

BezBroker

ЧАСТ 1 ОТ 2

Част 1: Разработка на API-то

Стъпка по стъпка изграждане на REST API с PHP 8, Slim 4 и MySQL.

Съдържание

1. Урок 01 — Преглед на API-то
2. Урок 02 — Подготовка на средата
3. Урок 03 — База данни (schema.sql)
4. Урок 04 — Slim 4 скелет
5. Урок 05 — Helper класове (Db, Json, Mapper)
6. Урок 06 — Middleware (CORS, JSON body, Auth)
7. Урок 07 — Auth контролер (register, login, me)
8. Урок 08 — Category контролер
9. Урок 09 — Estate контролер
10. Урок 10 — Photo контролер (качване на снимки)
11. Урок 11 — Offer и Message контролери
12. Урок 12 — Качване на Plesk + финален тест

Урок 01 — Преглед на API-то

Какво ще построим

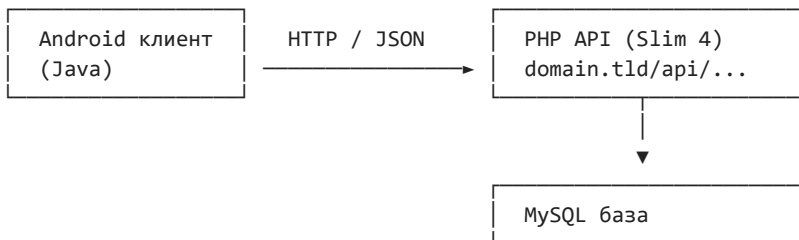
REST API за приложението BezBroker, на което Android клиентът се обажда. Стекът:

- **PHP 8.1+** като език
- **Slim 4** като микрофреймуърк (роутинг + middleware)
- **MySQL** като база данни
- **Composer** като пакетен мениджър
- **Plesk** като хостинг (опционално — работи и локално)

Какво ще поддържа API-то

Метод	Път	За какво
POST	/api/register	Регистрация на потребител
POST	/api/login	Login (връща токен)
GET	/api/me	Кой съм аз (по токен)
POST	/api/logout	Изтрива токена
GET	/api/categories	Списък с категории имоти
GET	/api/estates	Списък с имоти (с филтри)
GET	/api/estates/{id}	Детайл на имот
POST	/api/estates	Създаване на имот
DELETE	/api/estates/{id}	Изтриване на свой имот
GET	/api/estates/{id}/photos	Списък със снимки
POST	/api/estates/{id}/photos	Качване на снимки (multipart)
POST	/api/offers	Прави оферта
GET	/api/offers	Моите оферти
POST	/api/offers/{id}/state	Асепт / reject оферта
GET	/api/offers/{id}/messages	Съобщения в чат
POST	/api/offers/{id}/messages	Прави съобщение

Архитектура



Структура на проекта

В края на тези 12 урока, папката `api/` ще изглежда така:

```
api/
├── public/
│   ├── index.php          (входната точка – всичко минава оттук)
│   ├── .htaccess          (URL rewrite за Apache)
│   └── uploads/           (тук се пазят качените снимки)
├── src/
│   ├── App/
│   │   ├── AppFactoryHelper.php (регистрира middleware + routes)
│   │   ├── Db.php              (PDO singleton)
│   │   ├── Json.php            (helper за JSON отговори)
│   │   └── Mapper.php          (snake_case → camelCase)
│   ├── Middleware/
│   │   ├── CorsMiddleware.php
│   │   ├── JsonBodyMiddleware.php
│   │   └── AuthMiddleware.php
│   └── Controller/
│       ├── AuthController.php
│       ├── CategoryController.php
│       ├── EstateController.php
│       ├── PhotoController.php
│       ├── OfferController.php
│       └── MessageController.php
├── sql/
│   ├── schema.sql          (създава таблиците)
│   └── seed.sql             (примерни данни)
├── composer.json
├── .env.example
└── .gitignore
```

Конвенции

Запомнете тези три неща — ще ги виждаме във всеки следващ урок:

1. Всички API пътища започват с `/api/...` (например `/api/estates`).
2. JSON полета са в **camelCase** — `firstName`, `categoryId`, `isSeller` (за да съвпада с Android клиента).
3. Защитените пътища изискват `Authorization: Bearer<token>` header.

План на уроците

№	Тема
02	Подготовка на средата (PHP, Composer, MySQL)
03	База данни — <code>schema.sql</code>

№	Тема
04	Slim 4 скелет — composer.json и публична точка
05	Helper класове (Db, Json, Mapper)
06	Middleware — CORS, JSON body, Auth
07	Auth контролер (register, login, me)
08	Category контролер
09	Estate контролер
10	Photo upload контролер
11	Offer и Message контролери
12	Тест с curl + качване на Plesk

Готови сме

В Урок 02 започваме с подготовката на средата.

Урок 02 — Подготовка на средата

Цел

Да имаш инсталирани PHP 8.1+, Composer и локален MySQL, на които ще разработваш и тестваш API-то.

Какво ще инсталираме

Инструмент	За какво
PHP 8.1+	Изпълнява PHP кода
Composer	Сваля Slim и другите PHP зависимости
MySQL (XAMPP)	Базата данни
Postman или curl	За да тестваме endpoint-ите

Стъпка 1 — Инсталиране на PHP

Windows

1. Сваля PHP 8.1+ от <https://windows.php.net/download/> — изтегли "Thread Safe" версията като .zip.
2. Разархивирай в C:\php.
3. Добави C:\php към системната променлива PATH:
 - Settings → System → About → Advanced system settings → Environment Variables.
 - Намери Path → Edit → New → C:\php → OK.
4. В C:\php, преименувай php.ini-development → php.ini.
5. Отвори php.ini с текстов редактор и махни ; пред следните разширения:

```
extension=pdo_mysql
extension=mbstring
extension=openssl
extension=fileinfo
```

Проверка

Затвори и отвори отново терминала, после:

```
php -v
```

Очакваш нещо като:

```
PHP 8.2.12 (cli) (built: Oct 24 2023 21:15:15) (ZTS Visual C++ 2019 x64)
```

Стъпка 2 — Инсталиране на Composer

Сваля и стартирай инсталатора от <https://getcomposer.org/Composer-Setup.exe>.

При инсталацията той ще намери PHP-то ти автоматично, ако сложиш PATH-а в Стъпка 1.

Проверка

```
composer -V
```

Очакваш:

```
Composer version 2.6.5 2023-10-06 10:11:52
```

Стъпка 3 — Локална MySQL база

Най-лесно с **XAMPP** (включва Apache, MySQL и phpMyAdmin в един пакет):

1. Свали XAMPP от <https://www.apachefriends.org/>.
2. Инсталирай в C:\xampp.
3. Стартирай XAMPP Control Panel.
4. Натисни **Start** до "MySQL" (Apache не е нужен — ние ще ползваме PHP вградения сървър за API-то).

Стъпка 4 — Създаване на базата

Отвори <http://localhost/phpmyadmin> в браузъра.

1. Sidebar → New → име bezbroker → Collation utf8mb4_unicode_ci → **Create**.
2. След създаване, отиди в раздел **Privileges** на тази база:
 - **Add user account**.
 - User name: bezbroker
 - Host name: localhost
 - Password: bezbroker (или каквото предпочиташ — запиши го).
 - Database for user account: ☐ "Grant all privileges on database bezbroker".
 - Натисни **Go**.

Стъпка 5 — Postman или curl за тестване

curl е инсталиран по подразбиране в съвременен Windows 10/11. Провери:

```
curl --version
```

Ако предпочиташ GUI, свали **Postman** от <https://www.postman.com/downloads/> — безплатна версия е достатъчна.

Стъпка 6 — Папка за проекта

Решаваме къде ще живее API-то. Препоръчвам да е до Android проекта, в `api/` подпапка:

```
BezBroker/  
├─ app/           ← Android проектът  
├─ api/           ← ТУК ще пишем  
└─ tutorial/
```

В терминала:

```
cd C:\work\Teaching\Android\02\BezBroker-master  
mkdir api  
cd api
```

Цялата работа от следващите уроци ще е вътре в `api/`.

Готови сме

Готов си. В Урок 03 ще създадем базата данни.

Често срещани проблеми

Симптом	Причина
'php' is not recognized	Не си добавил C:\php към PATH, или не си рестартирал терминала.
'composer' is not recognized	Инсталацията на Composer не е приключила успешно — пусни инсталатора пак.
Connection refused при достъп до localhost/phpmyadmin	XAMPP MySQL не е стартиран. Натисни Start.
mysqli extension not loaded в phpMyAdmin	В php.ini махни ; пред extension=mysqli.

Урок 03 — База данни (schema.sql)

Цел

Да създадем 7-те таблици, които API-то ще ползва, плюс начални 5 категории за имоти.

Какво ще създадем

Таблица	Какво пази
user	Потребителски акаунти (паролите са bcrypt хешове)
token	Активни сесии — по една на login
category	Категории имоти (Apartment, House, ...)
estate	Обявите за имоти
estate_photo	Пътищата към качените снимки
offer	Оферти за имот
message	Съобщения в чата за конкретна оферта

Стъпка 1 — Папка sql/

В api/ създай:

```
cd api
mkdir sql
```

Стъпка 2 — schema.sql

Създай нов файл api/sql/schema.sql със следното съдържание:

```
-- BezBroker — MySQL schema

SET FOREIGN_KEY_CHECKS = 0;

DROP TABLE IF EXISTS message;
DROP TABLE IF EXISTS offer;
DROP TABLE IF EXISTS estate_photo;
DROP TABLE IF EXISTS estate;
DROP TABLE IF EXISTS category;
DROP TABLE IF EXISTS token;
DROP TABLE IF EXISTS user;

SET FOREIGN_KEY_CHECKS = 1;

CREATE TABLE user (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  email       VARCHAR(190) NOT NULL UNIQUE,
  password    VARCHAR(255) NOT NULL,
  phone       VARCHAR(40)  NULL,
  role        VARCHAR(20)  NOT NULL DEFAULT 'buyer',
  first_name  VARCHAR(100) NULL,
  last_name   VARCHAR(100) NULL,
  created_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP
```

```

) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_unicode_ci;

CREATE TABLE token (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  user_id     INT          NOT NULL,
  token       VARCHAR(80)  NOT NULL UNIQUE,
  created_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_token_user FOREIGN KEY (user_id)
    REFERENCES user(id) ON DELETE CASCADE
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_unicode_ci;

CREATE TABLE category (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  name        VARCHAR(100) NOT NULL UNIQUE,
  created_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_unicode_ci;

CREATE TABLE estate (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  category_id INT          NOT NULL,
  title       VARCHAR(200) NOT NULL,
  description  TEXT        NULL,
  city        VARCHAR(100) NOT NULL,
  price       DOUBLE       NOT NULL,
  area        DOUBLE       NOT NULL,
  address     VARCHAR(255) NULL,
  lat         DOUBLE       NULL,
  lont        DOUBLE       NULL,
  user_id     INT          NOT NULL,
  created_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_estate_category FOREIGN KEY (category_id)
    REFERENCES category(id),
  CONSTRAINT fk_estate_user FOREIGN KEY (user_id)
    REFERENCES user(id)
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_unicode_ci;

CREATE TABLE estate_photo (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  estate_id   INT          NOT NULL,
  file_name   VARCHAR(255) NOT NULL,
  created_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_photo_estate FOREIGN KEY (estate_id)
    REFERENCES estate(id) ON DELETE CASCADE
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_unicode_ci;

CREATE TABLE offer (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  sum         DOUBLE       NOT NULL,
  offered_by_user INT      NOT NULL,
  offered_for_estate INT    NOT NULL,
  state       VARCHAR(20)  NOT NULL DEFAULT 'pending',
  created_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_offer_user FOREIGN KEY (offered_by_user)
    REFERENCES user(id),
  CONSTRAINT fk_offer_estate FOREIGN KEY (offered_for_estate)
    REFERENCES estate(id) ON DELETE CASCADE
) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_unicode_ci;

CREATE TABLE message (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  text        TEXT        NOT NULL,
  from_user   INT          NOT NULL,
  to_user     INT          NOT NULL,
  offer_id    INT          NOT NULL,
  created_at  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_msg_from FOREIGN KEY (from_user) REFERENCES user(id),

```

```

        CONSTRAINT fk_msg_to    FOREIGN KEY (to_user)    REFERENCES user(id),
        CONSTRAINT fk_msg_offer FOREIGN KEY (offer_id)   REFERENCES offer(id) ON DELETE CASCADE
    ) ENGINE = InnoDB DEFAULT CHARSET = utf8mb4 COLLATE = utf8mb4_unicode_ci;

-- Семена данни: 5 категории
INSERT INTO category (name) VALUES
    ('Apartment'),
    ('House'),
    ('Land'),
    ('Office'),
    ('Garage');
```

Какво да забележим

- InnoDB е engine-ът, който поддържа foreign keys и транзакции.
- utf8mb4 — поддържа емоџи и всички unicode символи, включително кирилица.
- role VARCHAR(20) пази 'buyer' или 'seller'. Защо не bool? Защото утре може да добавим 'admin'.
- token е отделна таблица, не колона в user. Така един потребител може да е логнат от няколко устройства едновременно — всеки login прави нов token.
- ON DELETE CASCADE — ако изтрием имот, всичките му снимки, оферти и съобщения изчезват автоматично.
- lat и lont са DOUBLE NULL — не всеки имот има координати.

Стъпка 3 — Импорт в локалната база

Имаш два варианта:

Вариант А — phpMyAdmin

1. Отвори <http://localhost/phpmyadmin>.
2. Sidebar → избери базата bezbroker.
3. Раздел **Import** → **Choose File** → `api/sql/schema.sql` → **Go**.

Вариант В — Команден ред

```
mysql -u bezbroker -p bezbroker < api/sql/schema.sql
```

Той ще пита за парола.

Стъпка 4 — Проверка

В phpMyAdmin, в sidebar-а под bezbroker, трябва да виждаш 7 таблици:

```

bezbroker
├── category      (5 реда)
├── estate        (0 реда)
├── estate_photo  (0 реда)
├── message       (0 реда)
├── offer         (0 реда)
├── token         (0 реда)
└── user         (0 реда)
```

Натисни на category → виждаш 5-те категории.

Готови сме

В Урок 04 започваме PHP кода.

Често срещани грешки

Симптом	Причина
ERROR 1071: Specified key was too long	Старо MySQL (< 5.7.7) без InnoDB row format. Обнови MySQL или промени varchar дължините.
ERROR 1452: Cannot add or update a child row	Опитваш да създадеш ред с foreign key, който сочи към несъществуващ родител.
Access denied for user	Сгрешена парола за потребителя bezbroker, или потребителят няма права.

Урок 04 — Slim 4 скелет

Цел

Да инициализираме Composer проекта, да изтеглим Slim 4 и да направим входната точка `public/index.php`, която отговаря с health check.

Какво ще създадем

Файл	Какво прави
<code>composer.json</code>	Декларира зависимостите (Slim, PSR-7, dotenv)
<code>.env.example</code>	Шаблон за конфигурацията
<code>.gitignore</code>	Каква да не се качи в Git
<code>public/index.php</code>	Входната точка — всички заявки минават оттук
<code>public/.htaccess</code>	Apache rewrite правила

Стъпка 1 — `composer.json`

В `api/`, създай `composer.json`:

```
{
  "name": "uni/bezbroker-api",
  "description": "BezBroker — teaching API (Slim 4 + MySQL)",
  "type": "project",
  "require": {
    "php": ">=8.1",
    "slim/slim": "^4.13",
    "slim/psr7": "^1.6",
    "vlucas/phpdotenv": "^5.6"
  },
  "autoload": {
    "psr-4": {
      "BezBroker\\": "src/"
    }
  },
  "config": {
    "sort-packages": true
  }
}
```

Какво да забележим:

- `slim/slim` — самият framework.
- `slim/psr7` — PSR-7 имплементация на Request/Response обектите.
- `vlucas/phpdotenv` — зарежда `.env` файла в `$_ENV`.
- `"BezBroker\\": "src/"` е PSR-4 autoloader. Означава: всеки клас в namespace `BezBroker\\X\\Y` се търси в файл `src/X/Y.php`. Автоматично, без `require_once`.

Стъпка 2 — Инсталация на зависимостите

```
cd api
composer install
```

След няколко секунди ще се появи папка `vendor/` с библиотеките. Готови сме.

Стъпка 3 — `.env.example`

Създай `.env.example` (шаблон, който качваме в Git):

```
DB_HOST=localhost
DB_PORT=3306
DB_NAME=bezbroker
DB_USER=bezbroker
DB_PASS=changeme

UPLOADS_DIR=uploads

# Публичният адрес на API-то — ползва се за URL-и към снимки.
# Локално (Android emulator): http://10.0.2.2:8000
# Plesk production:           https://your-domain.tld
PUBLIC_BASE_URL=http://10.0.2.2:8000
```

Копирай го като `.env` (без `.example`) и попълни реалните данни за твоята локална база:

```
copy .env.example .env
```

Отвори `.env` и сложи паролата от Урок 02.

Стъпка 4 — `.gitignore`

```
/vendor/
/.env
/public/uploads/
.DS_Store
Thumbs.db
```

Защо: `vendor/` се изтегля от `composer install`, `.env` съдържа пароли, `uploads/` се пълни от потребители.

Стъпка 5 — `public/index.php`

Това е входната точка. Когато заявка дойде, Apache (или PHP вграденият сървър) сервира **този** файл.

```
mkdir -p public
```

Създай `public/index.php`:

```
<?php
declare(strict_types=1);

use Dotenv\Dotenv;
use Slim\Factory\AppFactory;

require __DIR__ . '/../vendor/autoload.php';

// Зареждаме .env от папка api/
Dotenv::createImmutable(__DIR__ . '/../')->safeLoad();

$app = AppFactory::create();

// Health check route
$app->get('/', function ($req, $res) {
    $res->getBody()->write(json_encode(['name' => 'BezBroker API', 'ok' => true]));
    return $res->withHeader('Content-Type', 'application/json');
});

$app->run();
```

Какво се случва, ред по ред:

Ред	Какво прави
<code>declare(strict_types=1)</code>	По-стриктна типизация — <code>int</code> параметър не приема <code>string</code> .
<code>require .../vendor/autoload.php</code>	Composer autoloader-ът; зарежда класове по нужда.
<code>Dotenv::createImmutable(...)->safeLoad()</code>	Чете <code>.env</code> в <code>\$_ENV</code> . <code>safeLoad</code> не хвърля грешка ако файлът липсва.
<code>AppFactory::create()</code>	Прави Slim приложението.
<code>\$app->get('/', fn(...) => ...)</code>	Регистрира GET / route.
<code>\$res->withHeader(...)</code>	Slim Response е immutable — <code>withHeader</code> връща нов обект, не модифицира оригиналния.
<code>\$app->run()</code>	Стартира — чете заявката, минава през middleware-ите, изпълнява route handler-a, връща отговор.

Стъпка 6 — public/.htaccess

Apache по подразбиране ще опита да сервира файл `bezbroker.com/api/estates` като файл с име `estates`. Ние искаме всичко да отива до `index.php`. Затова правим правило за `rewrite`:

Създай `public/.htaccess`:

```
RewriteEngine On

# Серирай реални файлове (например качените снимки) директно.
RewriteCond %{REQUEST_FILENAME} -f [OR]
RewriteCond %{REQUEST_FILENAME} -d
RewriteRule ^ - [L]

# Иначе пращай всичко към index.php
RewriteRule ^ index.php [QSA,L]
```

Обяснение:

- Първото правило: ако заявката е към реален файл (`snimka.jpg`) или папка, сервирай го директно.
- Второто: иначе — `index.php` поема.
- `[QSA, L]` — Query String Append + Last (спри обработката).

Стъпка 7 — Стартиране на сървъра локално

PHP има вграден сървър — за разработване е идеален.

```
cd api
php -S 0.0.0.0:8000 -t public
```

Какво значат флаговете:

- `-S 0.0.0.0:8000` — слушай на всички мрежови интерфейси, порт 8000.
- `-t public` — document root е папка `public/`.

Отвори в браузъра `http://localhost:8000/` — очаквай:

```
{"name": "BezBroker API", "ok": true}
```

Ако виждаш това — Slim работи!

***Бележка:** PHP вграденият сървър не чете .htaccess (защото не е Apache). Но за тестване ни стига — Slim сам си се грижи за роутинга. .htaccess-ът ще влезе в употреба чак на Plesk.*

Стъпка 8 — bezbroker.com/anything → 404 със JSON

Slim по подразбиране връща HTML страница при 404. Не е удобно за API. Това ще оправим в Урок 06 с custom error handler.

Засега тестваме че `http://localhost:8000/nonexistent` връща Slim default 404 (HTML страница) — нормално е.

Готови сме

В Урок 05 пишем helper класове за база данни и JSON отговори.

Често срещани грешки

Симптом	Причина
Class "Dotenv\Dotenv" not found	Не си пуснал <code>composer install</code> .
Connection refused на localhost:8000	Друг процес ползва порт 8000. Сменя на 8001: <code>php -S 0.0.0.0:8001 -t public.</code>
Виждаш source кода на PHP вместо изхода	Без PHP инсталиран. Виж Урок 02.
Could not open input file: public/index.php	Пуснал си <code>php -S</code> от грешна папка. Трябва от <code>api/</code> .

Урок 05 — Helper класове (Db, Json, Mapper)

Цел

Преди да пишем endpoint-и, трябва ни:

- Връзка към MySQL (PDO).
- Лесен начин да върнем JSON отговор.
- Преобразуване от snake_case (база) → camelCase (JSON за Android клиента).

Тези три неща ще ги слагаме в namespace BezBroker\App.

Файлове, които създаваме

```
api/src/App/  
├─ Db.php  
├─ Json.php  
└─ Mapper.php
```

Стъпка 1 — Създаване на папките

```
cd api  
mkdir -p src/App
```

PSR-4 autoloader-ът ще намира класовете автоматично:

- BezBroker\App\Db → src/App/Db.php
- BezBroker\App\Json → src/App/Json.php
- и т.н.

Стъпка 2 — Db.php (PDO singleton)

Създай api/src/App/Db.php:

```
<?php  
declare(strict_types=1);  
  
namespace BezBroker\App;  
  
use PDO;  
  
final class Db  
{  
    private static ?PDO $pdo = null;  
  
    public static function pdo(): PDO  
    {  
        if (self::$pdo !== null) {  
            return self::$pdo;  
        }  
        $host = $_ENV['DB_HOST'] ?? 'localhost';  
        $port = $_ENV['DB_PORT'] ?? '3306';  
        $name = $_ENV['DB_NAME'] ?? 'bezbroker';  
        $user = $_ENV['DB_USER'] ?? 'root';  
        $pass = $_ENV['DB_PASS'] ?? '';  
  
        $dsn = "mysql:host=$host;port=$port;dbname=$name;charset=utf8mb4";  
        self::$pdo = new PDO($dsn, $user, $pass, [  
            PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,  
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,  
        ]  
    )  
}
```

```

        PDO::ATTR_EMULATE_PREPARES => false,
    ]);
    return self::$pdo;
}
}

```

Какво да забележим:

- `final class` — не може да се наследява. Сигнал че няма смисъл от подкласове.
- `private static ?PDO $pdo` — пазим една и съща връзка за целия request. Това е singleton pattern.
- `Db::pdo()` — статичен метод; ползва се навсякъде като `Db::pdo()->prepare(...)`.
- **PDO options:**
 - `ERRMODE_EXCEPTION` — при SQL грешка хвърля `PDOException` вместо да връща `false`. Така грешките не се пропускат.
 - `FETCH_ASSOC` — `fetch()` връща асоциативен масив (с имена на колони) вместо тъп numeric array.
 - `EMULATE_PREPARES = false` — реални prepared statements на ниво MySQL, не emulated. По-сигурно срещу SQL injection.

Стъпка 3 — Употреба на Db в index.php

Бързо тест: модифицирай временно `public/index.php` за да види дали връзката работи. Добави нов route:

```

use BezBroker\App\Db;

// ...

$app->get('/db-test', function ($req, $res) {
    $count = Db::pdo()->query('SELECT COUNT(*) FROM category')->fetchColumn();
    $res->getBody()->write(json_encode(['categories' => $count]));
    return $res->withHeader('Content-Type', 'application/json');
});

```

Стартирай `php -S 0.0.0.0:8000 -t public` и отвори `http://localhost:8000/db-test`. Очаквай:

```

{"categories":5}

```

Ако виждаш това — връзката работи. Може да изтриеш `/db-test` route-а след това.

Стъпка 4 — Json.php (отговори)

В цялото API ще искаме да връщаме JSON. Вместо да повтаряме:

```

$res->getBody()->write(json_encode($data));
return $res->withHeader('Content-Type', 'application/json');

```

навсякъде, правим helper. Създай `api/src/App/Json.php`:

```

<?php
declare(strict_types=1);

namespace BezBroker\App;

use Psr\Http\Message\ResponseInterface;

final class Json
{
    public static function write(
        ResponseInterface $response,
        mixed $data,
        int $status = 200
    ): ResponseInterface {
        $response->getBody()->write(

```

```

        json_encode($data, JSON_UNESCAPED_UNICODE | JSON_UNESCAPED_SLASHES)
    );
    return $response
        ->withStatus($status)
        ->withHeader('Content-Type', 'application/json');
}

public static function error(
    ResponseInterface $response,
    string $message,
    int $status = 400
): ResponseInterface {
    return self::write($response, [
        'error' => true,
        'message' => $message,
    ], $status);
}
}

```

Какво да забележим:

- `JSON_UNESCAPED_UNICODE` — кирилицата не става а... а остава буквена. По-малки JSON отговори и четими.
- `JSON_UNESCAPED_SLASHES` — URL-и не стават `http:\\\\...` а просто `http://...`
- `Json::error(...)` — стандартен формат за грешки: `{"error": true, "message": "..."}.`

Стъпка 5 — Употреба на Json

Пренаписваме health check route-а в `index.php`:

```

use BezBroker\App\Json;

$app->get('/', function ($req, $res) {
    return Json::write($res, ['name' => 'BezBroker API', 'ok' => true]);
});

```

Чисто.

Стъпка 6 — Mapper.php (snake_case → camelCase)

Базата ползва `first_name`, `category_id`, `created_at`. Но Android клиентът чете `firstName`, `categoryId`, `createdAt` (camelCase).

Опции:

1. Преименуваме колоните в базата — лошо, ще нарушим SQL стандартите.
2. Преименуваме при четене с `AS aliases` — много код във всеки `SELECT`.
3. Помощник, който преобразува масивите преди да ги върнем — **избираме това**.

Създай `api/src/App/Mapper.php`:

```

<?php
declare(strict_types=1);

namespace BezBroker\App;

final class Mapper
{
    public static function user(array $row): array
    {
        return [
            'id'          => (int) $row['id'],
            'email'        => $row['email'] ?? '',
            'phone'        => $row['phone'] ?? '',

```

```

        'firstName' => $row['first_name'] ?? '',
        'lastName' => $row['last_name'] ?? '',
        'role'      => $row['role'] ?? 'buyer',
        'isSeller'  => (($row['role'] ?? 'buyer') === 'seller'),
        'createdAt' => $row['created_at'] ?? null,
    ];
}

public static function estate(array $row, array $photoUrls = []): array
{
    return [
        'id'           => (int) $row['id'],
        'categoryId'   => (int) $row['category_id'],
        'categoryName' => $row['category_name'] ?? null,
        'title'        => $row['title'] ?? '',
        'description'   => $row['description'] ?? '',
        'city'         => $row['city'] ?? '',
        'price'        => (float) ($row['price'] ?? 0),
        'area'         => (float) ($row['area'] ?? 0),
        'address'      => $row['address'] ?? '',
        'lat'          => isset($row['lat']) && $row['lat'] !== null ? (float) $row['lat'] : null,
        'lont'         => isset($row['lont']) && $row['lont'] !== null ? (float) $row['lont'] : null,
        'userId'       => (int) $row['user_id'],
        'createdAt'    => $row['created_at'] ?? null,
        'photos'       => $photoUrls,
        'ownerEmail'   => $row['owner_email'] ?? null,
        'ownerPhone'   => $row['owner_phone'] ?? null,
        'ownerFirstName' => $row['owner_first_name'] ?? null,
        'ownerLastName' => $row['owner_last_name'] ?? null,
    ];
}

public static function offer(array $row): array
{
    return [
        'id'           => (int) $row['id'],
        'sum'          => (float) ($row['sum'] ?? 0),
        'offeredByUser' => (int) ($row['offered_by_user'] ?? 0),
        'offeredForEstate' => (int) ($row['offered_for_estate'] ?? 0),
        'state'        => $row['state'] ?? 'pending',
        'createdAt'    => $row['created_at'] ?? null,
        'estateTitle'  => $row['estate_title'] ?? null,
        'estateOwnerId' => isset($row['estate_owner_id'])
            ? (int) $row['estate_owner_id'] : null,
        'buyerName'    => trim(($row['buyer_first_name'] ?? '')
            . ' '
            . ($row['buyer_last_name'] ?? '')),
        'sellerName'   => trim(($row['seller_first_name'] ?? '')
            . ' '
            . ($row['seller_last_name'] ?? '')),
    ];
}

public static function message(array $row): array
{
    return [
        'id'           => (int) $row['id'],
        'text'         => $row['text'] ?? '',
        'fromUser'     => (int) ($row['from_user'] ?? 0),
        'toUser'       => (int) ($row['to_user'] ?? 0),
        'offerId'      => (int) ($row['offer_id'] ?? 0),
        'createdAt'    => $row['created_at'] ?? null,
    ];
}

public static function category(array $row): array

```

```

{
    return [
        'id' => (int) $row['id'],
        'name' => $row['name'] ?? '',
    ];
}
}

```

Какво да забележим:

- `(int)` и `(float)` **cast-ове** — PDO връща числа като string в някои случаи. Cast-ваме за да получи Android правилни типове.
- `'isSeller' => $role === 'seller'` — преобразуваме string-ово поле в boolean за клиента.
- `'photos' => $photoUrls` — `Mapper::estate` приема готов масив с URL-и към снимки като втори аргумент. Естейт контролерът ще ги изчисли отделно (Урок 09).

Готови сме

Сега имаме:

- `Db::pdo()` за заявки.
- `Json::write($res, $data)` за JSON отговор.
- `Mapper::user($row)`, `Mapper::estate(...)`, и т.н. за форматиране.

В Урок 06 добавяме middleware.

Често срещани грешки

Симптом	Причина
Class "BezBroker\App\Db" not found	PSR-4 autoloader търси <code>src/App/Db.php</code> . Провери че пътя е точно такъв (case-sensitive!).
SQLSTATE[HY000] [2002] No connection could be made	<code>.env</code> сочи към грешен <code>DB_HOST</code> или MySQL не е стартиран.
SQLSTATE[HY000] [1045] Access denied	Грешна парола в <code>.env</code> .
SQLSTATE[42S02]: Base table or view not found	<code>schema.sql</code> не е импортнат.

Урок 06 — Middleware (CORS, JSON body, Auth)

Цел

Middleware е код, който се изпълнява **преди** и **след** всеки request — за да може да направим неща като:

- Добавяме CORS headers за всички отговори (за да може браузър да достигне API-то).
- Парсваме JSON в request тялото в `getParsedBody()`.
- Проверяваме Authorization header-а за защитените route-ове.

Файлове, които ще създадем

```
api/src/Middleware/  
├─ CorsMiddleware.php  
├─ JsonBodyMiddleware.php  
└─ AuthMiddleware.php
```

Стъпка 1 — Папка Middleware/

```
cd api  
mkdir -p src/Middleware
```

Стъпка 2 — CorsMiddleware

CORS (Cross-Origin Resource Sharing) — браузърите блокират AJAX заявки към различен домейн, освен ако сървърът не каже изрично "OK".

За мобилно приложение CORS не е задължителен — Android клиентът не е браузър. Но е добра практика да го имаме.

Създай `api/src/Middleware/CorsMiddleware.php`:

```
<?php  
declare(strict_types=1);  
  
namespace BezBroker\Middleware;  
  
use Psr\Http\Message\ResponseInterface as Response;  
use Psr\Http\Message\ServerRequestInterface as Request;  
use Psr\Http\Server\MiddlewareInterface;  
use Psr\Http\Server\RequestHandlerInterface as Handler;  
  
final class CorsMiddleware implements MiddlewareInterface  
{  
    public function process(Request $request, Handler $handler): Response  
    {  
        $response = $handler->handle($request);  
        return $response  
            ->withHeader('Access-Control-Allow-Origin', '*')  
            ->withHeader('Access-Control-Allow-Headers', 'Content-Type, Authorization')  
            ->withHeader('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, OPTIONS');  
    }  
}
```

Какво се случва:

Ред	Какво прави
<code>\$handler->handle(\$request)</code>	Извиква следващия middleware или route-a; връща готов отговор.
<code>withHeader('Access-Control-Allow-Origin', '*')</code>	Позволява заявки от всякакъв домейн (за разработване).
<code>withHeader('Access-Control-Allow-Headers', ...)</code>	Кои headers може клиентът да изпрати. Authorization ни трябва за токена.
<code>withHeader('Access-Control-Allow-Methods', ...)</code>	Кои HTTP методи са разрешени.

За production: Access-Control-Allow-Origin: * е твърде свободно. По-добре е `https://your-domain.tld (само твоя сайт)`. За учебни цели * е ОК.

Стъпка 3 — JsonBodyMiddleware

Slim има вграден `addBodyParsingMiddleware()`, който парсва JSON ако header-ът е `Content-Type: application/json`. Проблемът: клиенти често забравят header-a, и тогава `$request->getParsedBody()` връща `null`.

Този middleware попълва пропуска: ако тялото изглежда като JSON (започва с `{`), парсваме го независимо от header-a.

Създай `api/src/Middleware/JsonBodyMiddleware.php`:

```
<?php
declare(strict_types=1);

namespace BezBroker\Middleware;

use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;
use Psr\Http\Server\MiddlewareInterface;
use Psr\Http\Server\RequestHandlerInterface as Handler;

final class JsonBodyMiddleware implements MiddlewareInterface
{
    public function process(Request $request, Handler $handler): Response
    {
        if ($request->getParsedBody() === null) {
            $raw = (string) $request->getBody();
            if ($raw !== '' && str_starts_with(trim($raw), '{')) {
                $decoded = json_decode($raw, true);
                if (is_array($decoded)) {
                    $request = $request->withParsedBody($decoded);
                }
            }
        }
        return $handler->handle($request);
    }
}
```

Защо: без този middleware, при заявка без `Content-Type` header, контролерите ще получат `null` от `$request->getParsedBody()` и ще трябва ръчно да викат `json_decode($request->getBody())` навсякъде. По-чисто е тук.

Стъпка 4 — AuthMiddleware

За route-ове като POST /api/estates или GET /api/me, клиентът праща header:

```
Authorization: Bearer abc123def456...
```

(Бележка: Android клиентът пропуска интервала между Bearer и токена — затова приемаме и двата формата.)

Middleware-ът:

1. Чете Authorization header-а.
2. Маха префикса Bearer (с или без интервал).
3. Търси токена в таблица token JOIN user.
4. Ако не е намерен → връща 401.
5. Иначе — закача user обекта като атрибут на request-а, така че контролерите имат достъп:
\$request->getAttribute('user').

Създай api/src/Middleware/AuthMiddleware.php:

```
<?php
declare(strict_types=1);

namespace BezBroker\Middleware;

use BezBroker\App\Db;
use BezBroker\App\Json;
use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;
use Psr\Http\Server\MiddlewareInterface;
use Psr\Http\Server\RequestHandlerInterface as Handler;
use Slim\Psr7\Response as SlimResponse;

final class AuthMiddleware implements MiddlewareInterface
{
    public function process(Request $request, Handler $handler): Response
    {
        $header = $request->getHeaderLine('Authorization');
        if ($header === '') {
            return Json::error(new SlimResponse(), 'Missing Authorization header', 401);
        }

        // Маха префикса "Bearer" (със или без интервал)
        $token = $header;
        if (stripos($token, 'Bearer') === 0) {
            $token = trim(substr($token, 6));
        }
        if ($token === '') {
            return Json::error(new SlimResponse(), 'Empty token', 401);
        }

        $stmt = Db::pdo()->prepare(
            'SELECT u.* FROM user u
            JOIN token t ON t.user_id = u.id
            WHERE t.token = :t LIMIT 1'
        );
        $stmt->execute([':t' => $token]);
        $user = $stmt->fetch();
        if (!$user) {
            return Json::error(new SlimResponse(), 'Invalid token', 401);
        }

        unset($user['password']);
        $request = $request
            ->withAttribute('user', $user)
```

```

        ->withAttribute('token', $token);

    return $handler->handle($request);
}
}

```

Какво да забележим:

- `stripos($token, 'Bearer') === 0` — case-insensitive проверка дали започва с "Bearer".
- `substr($token, 6) + trim()` — маха "Bearer" (6 символа) и оптимизирано се справя с или без интервала.
- **JOIN-ЪТ** — една заявка проверява И токена И зарежда user-a.
- `unset($user['password'])` — не искаме хеша на паролата да изтича към контролерите.
- `->withAttribute('user', $user)` — атрибутите на request са начин за middleware да предаде данни към следващия handler.

Стъпка 5 — Регистриране на middleware-ите в index.php

В `public/index.php`, под `Dotenv::createImmutable(...)->safeLoad();`, добави:

```

use BezBroker\Middleware\CorsMiddleware;
use BezBroker\Middleware\JsonBodyMiddleware;

```

И след `$app = AppFactory::create();`, преди първия route, добави:

```

$app->addBodyParsingMiddleware();
$app->add(new JsonBodyMiddleware());
$app->add(new CorsMiddleware());

// Custom error handler (върща JSON вместо HTML страница)
$errorMiddleware = $app->addErrorMiddleware(false, true, true);
$errorMiddleware->setDefaultErrorHandler(function ($request, \Throwable $exception) {
    $status = 500;
    if ($exception instanceof \Slim\Exception\HttpNotFoundException) $status = 404;
    if (method_exists($exception, 'getStatusCode')
        && $exception->getStatusCode() >= 400 && $exception->getStatusCode() < 600) {
        $status = $exception->getStatusCode();
    }
    $payload = ['error' => true, 'message' => $exception->getMessage() ?: 'Server error'];
    $res = (new \Slim\Psr7\Response())->withStatus($status);
    $res->getBody()->write(json_encode($payload, JSON_UNESCAPED_UNICODE));
    return $res->withHeader('Content-Type', 'application/json');
});

```

Какво да забележим:

- **Редът на `$app->add(...)`** е важен. Middleware-и се изпълняват в **обратен ред** от регистрацията. CORS е добавен последен, значи се изпълнява пръв — обвива всички останали.
- `addErrorMiddleware(false, true, true)` — `display errors = false` (не показваме stack trace на клиента), `log errors = true`, `log details = true`.
- **JSON error handler** — за всяка грешка връща JSON с `{error: true, message: ...}`.

Стъпка 6 — OPTIONS route (за CORS preflight)

Когато браузър прави POST с custom headers, той първо праща OPTIONS заявка ("preflight"), за да провери дали CORS позволява. Slim не може сам да отговори на OPTIONS, така че добавяме fallback:

В `index.php`, преди `$app->run();`:

```

$app->options('/{routes:.+}', fn($req, $res) => $res);

```

`/ {routes: .+}` означава "всички пътища". OPTIONS route отговаря с празно тяло — CORS middleware-ът автоматично слага headers-ите.

Стъпка 7 — Тест

Стартирай сървъра:

```
php -S 0.0.0.0:8000 -t public
```

Тестове:

```
# 1. Health check (трябва да работи)
curl http://localhost:8000/
```

```
# 2. 404 трябва да върне JSON, не HTML
curl http://localhost:8000/nonexistent
```

```
# Очаквай:
# {"error":true,"message":"Not found."}
```

```
# 3. Защитен endpoint без токен (още няма такъв ред, но за тест)
curl -i http://localhost:8000/api/me
# Очаквай 404 засега — /api/me ще го направим в следващия урок.
```

Готови сме

В Урок 07 пишем първия истински endpoint — Auth контролер с register, login и me.

Често срещани грешки

Симптом	Причина
404 връща HTML страница	Не си регистрирал custom error handler.
Class "BezBroker\Middleware\..." not found	Composer autoloader-ът все още кешира стария списък. Пусни <code>composer dump-autoload</code> .
CORS error в браузър	Access-Control-Allow-Origin не е сложен — провери че CorsMiddleware е регистриран.
Argument #1 must be of type Request	Middleware-ите трябва да implement-ват MiddlewareInterface с точно тази сигнатура.

Урок 07 — Auth контролер (register, login, me)

Цел

Първият истински контролер. Покрива:

- POST /api/register — нов потребител, връща токен.
- POST /api/login — съществуващ потребител, връща токен.
- GET /api/me — кой съм аз (защитен).
- POST /api/logout — изтрива токена (защитен).

Стъпка 1 — Папка Controller/

```
cd api
mkdir -p src/Controller
```

Стъпка 2 — AuthController.php

Създай api/src/Controller/AuthController.php:

```
<?php
declare(strict_types=1);

namespace BezBroker\Controller;

use BezBroker\App\Db;
use BezBroker\App\Json;
use BezBroker\App\Mapper;
use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;

final class AuthController
{
    public function register(Request $request, Response $response): Response
    {
        $body      = (array) ($request->getParsedBody() ?? []);
        $email      = trim((string) ($body['email'] ?? ''));
        $password   = (string) ($body['password'] ?? '');
        $firstName  = trim((string) ($body['firstName'] ?? ''));
        $lastName   = trim((string) ($body['lastName'] ?? ''));
        $phone      = trim((string) ($body['phone'] ?? ''));
        $isSeller   = (bool) ($body['isSeller'] ?? false);

        if ($email === '' || $password === '') {
            return Json::error($response, 'Email and password are required.', 422);
        }
        if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
            return Json::error($response, 'Invalid email.', 422);
        }
        if (strlen($password) < 6) {
            return Json::error($response, 'Password must be at least 6 characters.', 422);
        }

        $pdo = Db::pdo();
        $exists = $pdo->prepare('SELECT 1 FROM user WHERE email = :e');
        $exists->execute([':e' => $email]);
        if ($exists->fetchColumn()) {
            return Json::error($response, 'Email is already registered.', 409);
        }

        $hash = password_hash($password, PASSWORD_BCRYPT);
```

```

        $pdo->prepare(
            'INSERT INTO user (email, password, phone, role, first_name, last_name)
            VALUES (:e, :p, :ph, :r, :fn, :ln)'
        )->execute([
            ':e' => $email,
            ':p' => $hash,
            ':ph' => $phone,
            ':r' => $isSeller ? 'seller' : 'buyer',
            ':fn' => $firstName,
            ':ln' => $lastName,
        ]);

        $userId = (int) $pdo->lastInsertId();
        return Json::write($response, [
            'token' => $this->createToken($userId),
            'user' => Mapper::user($this->loadUser($userId)),
        ], 201);
    }

    public function login(Request $request, Response $response): Response
    {
        $body      = (array) ($request->getParsedBody() ?? []);
        $email      = trim((string) ($body['email'] ?? ''));
        $password    = (string) ($body['password'] ?? '');

        if ($email === '' || $password === '') {
            return Json::error($response, 'Email and password are required.', 422);
        }

        $stmt = Db::pdo()->prepare('SELECT * FROM user WHERE email = :e LIMIT 1');
        $stmt->execute([':e' => $email]);
        $user = $stmt->fetch();

        if (!$user || !password_verify($password, $user['password'])) {
            return Json::error($response, 'Invalid email or password.', 401);
        }

        return Json::write($response, [
            'token' => $this->createToken((int) $user['id']),
            'user' => Mapper::user($user),
        ]);
    }

    public function logout(Request $request, Response $response): Response
    {
        $token = (string) $request->getAttribute('token');
        Db::pdo()->prepare('DELETE FROM token WHERE token = :t')->execute([':t' => $token]);
        return Json::write($response, ['ok' => true]);
    }

    public function me(Request $request, Response $response): Response
    {
        return Json::write($response, [
            'user' => Mapper::user($request->getAttribute('user')),
        ]);
    }

    // -----
    private function createToken(int $userId): string
    {
        $token = bin2hex(random_bytes(32));
        Db::pdo()->prepare('INSERT INTO token (user_id, token) VALUES (:u, :t)')
            ->execute([':u' => $userId, ':t' => $token]);
        return $token;
    }
}

```

```

private function loadUser(int $userId): array
{
    $stmt = Db::pdo()->prepare('SELECT * FROM user WHERE id = :id');
    $stmt->execute([':id' => $userId]);
    return $stmt->fetch() ?: [];
}
}

```

Концепциите ред по ред:

Тема	Какво да обясним
(array) (\$request->getParsedBody() ?? [])	Ако тялото е празно/невалидно, ползваме празен масив, за да не crash-нем при \$body['email'].
trim() и (string) cast	Защитаваме се от null или whitespace вход.
filter_var(\$email, FILTER_VALIDATE_EMAIL)	Вградена PHP проверка за валиден email.
password_hash(\$password, PASSWORD_BCRYPT)	Прави bcrypt хеш — еднопосочна функция. Никога не пазим оригинална парола.
password_verify(\$password, \$user['password'])	Сравнява въведената парола с хеша.
bin2hex(random_bytes(32))	Генерира 64-знаков hex токен (256 бита). Криптографски сигурно.
Mapper::user(\$user)	Преобразуваме snake_case row → camelCase JSON.
Връщаме {token, user}	Точно това очаква Android клиентът.

Стъпка 3 — Регистриране на route-ите

В public/index.php, добави import-a:

```

use BezBroker\Controller\AuthController;
use BezBroker\Middleware\AuthMiddleware;

```

И регистрирай route-ите (преди \$app->run()):

```

// Public auth route-ове (без токен)
$app->group('/api', function ($g) {
    $g->post('/register', [AuthController::class, 'register']);
    $g->post('/login', [AuthController::class, 'login']);

    // Authenticated route-ове (с токен)
    $g->group('', function ($auth) {
        $auth->get('/me', [AuthController::class, 'me']);
        $auth->post('/logout', [AuthController::class, 'logout']);
    }->add(new AuthMiddleware()));
});

```

Какво се случва:

- \$app->group('/api', ...) — всички route-ове вътре получават префикс /api. Така register става /api/register.
- **Вложен** \$g->group('', ...) с ->add(new AuthMiddleware()) — само route-ите в този под-група минават през AuthMiddleware.

- `[AuthController::class, 'register']` — Slim ще инстанцира контролера и ще извика метода. Може и `function ($req, $res) { ... }` като callback, но контролерите са по-чисти.

Стъпка 4 — Тест с curl

Стартирай сървъра:

```
cd api
php -S 0.0.0.0:8000 -t public
```

Регистрация:

```
curl -X POST http://localhost:8000/api/register \
  -H "Content-Type: application/json" \
  -d '{"email":"ana@test.bg","password":"123456","firstName":"Ana","lastName":"Ivanova","isSeller":true}'
```

Очаквай:

```
{
  "token": "abc123def456...",
  "user": {
    "id": 1,
    "email": "ana@test.bg",
    "firstName": "Ana",
    "lastName": "Ivanova",
    "role": "seller",
    "isSeller": true,
    ...
  }
}
```

Запиши токена — ще ни трябва.

Login:

```
curl -X POST http://localhost:8000/api/login \
  -H "Content-Type: application/json" \
  -d '{"email":"ana@test.bg","password":"123456"}'
```

Очаквай: нов токен (различен от register-а — всеки login прави нов).

/api/me (защитен):

```
TOKEN="токенът от login"

curl http://localhost:8000/api/me \
  -H "Authorization: Bearer $TOKEN"
```

Очаквай: `{"user": {...}}`.

/api/me без токен:

```
curl -i http://localhost:8000/api/me
```

Очаквай:

```
HTTP/1.1 401 Unauthorized
{"error":true,"message":"Missing Authorization header"}
```

/api/me с грешен токен:

```
curl -i http://localhost:8000/api/me -H "Authorization: Bearer wrongtoken"
```

Очаквай 401 — "Invalid token".

Logout:

```
curl -X POST http://localhost:8000/api/logout \
  -H "Authorization: Bearer $TOKEN"

# {"ok":true}
```

След logout същия token повече не работи:

```
curl -i http://localhost:8000/api/me -H "Authorization: Bearer $TOKEN"
# HTTP/1.1 401 - Invalid token
```

Стъпка 5 — Проверка в базата

В phpMyAdmin → user → виждаш реда за Ана.

В token → виждаш активните токени (по 1 на login).

Сега Ана може да направи /api/login 3 пъти от 3 различни устройства — ще има 3 реда в token. Когато едно устройство направи /api/logout — само неговият token изчезва; другите 2 продължават да работят.

Готови сме

В Урок 08 ще направим category контролера (много прост).

Често срещани грешки

Симптом	Причина
409 при register с нов email	Може да си регистрирал друг потребител с този email. Виж user таблицата.
401 при login с правилна парола	Преди това си регистрирал с друга парола, паролата не съвпада. Регистрирай се пак с нов email или DELETE-ни user-а от phpMyAdmin.
Call to undefined function password_hash()	PHP < 5.5. Обнови до 8.1+.
random_bytes грешка	OpenSSL extension не е включен — провери php.ini.

Урок 08 — Category контролер

Цел

Най-простият контролер в проекта. Един endpoint, който връща списък с категории.

Защо отделен контролер за нещо толкова просто?

Можем да го сложим директно като closure в `index.php`:

```
$g->get('/categories', function ($req, $res) {
    $rows = Db::pdo()->query('SELECT * FROM category')->fetchAll();
    return Json::write($res, ['categories' => $rows]);
});
```

Но за тренировка и за консистентност с другите endpoint-и, го правим контролер. Освен това утре може да добавим `create/update/delete` на категории за `admin` потребител.

Стъпка 1 — CategoryController.php

Създай `api/src/Controller/CategoryController.php`:

```
<?php
declare(strict_types=1);

namespace BezBroker\Controller;

use BezBroker\App\Db;
use BezBroker\App\Json;
use BezBroker\App\Mapper;
use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;

final class CategoryController
{
    public function list(Request $request, Response $response): Response
    {
        $rows = Db::pdo()
            ->query('SELECT id, name FROM category ORDER BY name')
            ->fetchAll();

        return Json::write($response, [
            'categories' => array_map(fn($r) => Mapper::category($r), $rows),
        ]);
    }
}
```

Какво да забележим:

- `$pdo->query(...)` е по-кратко от `prepare + execute`, когато няма параметри.
- `->fetchAll()` — връща всички редове като масив.
- `array_map(fn($r) => Mapper::category($r), $rows)` — преобразуваме всеки ред. Лямбда нотация (PHP 7.4+).
- Връщаме обвито в `{categories: [...]}` — Android клиентът чете `body.optJSONArray("categories")`.

Стъпка 2 — Регистриране на route-a

В `public/index.php`, добави `import`:

```
use BezBroker\Controller\CategoryController;
```

В `$app->group('/api', ...)`, добави **извън** `authenticated` подгрупата (категориите са публични — всеки може да ги види):

```
$g->get('/categories', [CategoryController::class, 'list']);
```

Финалният фрагмент трябва да изглежда нещо подобно:

```
$app->group('/api', function ($g) {
    $g->post('/register', [AuthController::class, 'register']);
    $g->post('/login', [AuthController::class, 'login']);
    $g->get('/categories', [CategoryController::class, 'list']); // ← нов

    $g->group('', function ($auth) {
        $auth->get('/me', [AuthController::class, 'me']);
        $auth->post('/logout', [AuthController::class, 'logout']);
    }->add(new AuthMiddleware()));
});
```

Стъпка 3 — Тест с curl

```
curl http://localhost:8000/api/categories
```

Очаквай:

```
{
  "categories": [
    {"id": 1, "name": "Apartment"},
    {"id": 2, "name": "Garage"},
    {"id": 3, "name": "House"},
    {"id": 4, "name": "Land"},
    {"id": 5, "name": "Office"}
  ]
}
```

Категориите са подредени по име, **не** по `id` — заради `ORDER BY name` в SQL-a.

Опционално: Insert на собствена категория

Понеже `schema.sql` пуска само 5, ако искаш да добавиш още (например за тестване):

```
mysql -u bezbroker -p bezbroker -e "INSERT INTO category (name) VALUES ('Vacation home');"
```

Или през `phpMyAdmin` → `category` → `Insert`.

Готови сме

Получихме чувството за `pattern-a`:

1. Класът-контролер с метод-функция.
2. Регистриране в `route`-овете.
3. Тест с `curl`.

Всички следващи контролери са по същия шаблон, само с повече логика. В Урок 09 правим `Estate` контролера — там идва истинската работа.

Често срещани грешки

Симптом	Причина
Call to undefined method <code>query()</code>	<code>Db::pdo()</code> връща <code>null</code> — провери <code>.env</code>

Симптом	Причина
Празен масив {categories: []}	schema.sql не е изпълнен, или си изтрил INSERT-а.
500 Internal Server Error	Виж PHP лога. Най-често — typo в SQL-а.

Урок 09 — Estate контролер

Цел

Най-богатият контролер в проекта. Покрива:

- GET /api/estates — списък с филтри (city, categoryId, минимална/максимална цена и т.н.)
- GET /api/estates/{id} — детайл на един имот, включително име на собственика
- POST /api/estates — създаване (защитен)
- DELETE /api/estates/{id} — изтриване (защитен, само собственикът)

Стъпка 1 — EstateController.php

Създай api/src/Controller/EstateController.php:

```
<?php
declare(strict_types=1);

namespace BezBroker\Controller;

use BezBroker\App\Db;
use BezBroker\App\Json;
use BezBroker\App\Mapper;
use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;

final class EstateController
{
    public function list(Request $request, Response $response): Response
    {
        $q = $request->getQueryParams();
        $where = [];
        $params = [];

        if (!empty($q['city'])) {
            $where[] = 'e.city LIKE :city';
            $params[':city'] = '%' . $q['city'] . '%';
        }
        if (!empty($q['categoryId'])) {
            $where[] = 'e.category_id = :cat';
            $params[':cat'] = (int) $q['categoryId'];
        }
        if (!empty($q['minPrice'])) {
            $where[] = 'e.price >= :minP';
            $params[':minP'] = (float) $q['minPrice'];
        }
        if (!empty($q['maxPrice'])) {
            $where[] = 'e.price <= :maxP';
            $params[':maxP'] = (float) $q['maxPrice'];
        }
        if (!empty($q['minArea'])) {
            $where[] = 'e.area >= :minA';
            $params[':minA'] = (float) $q['minArea'];
        }
        if (!empty($q['userId'])) {
            $where[] = 'e.user_id = :uid';
            $params[':uid'] = (int) $q['userId'];
        }

        $sql = 'SELECT e.*, c.name AS category_name
FROM estate e
```

```

        JOIN category c ON c.id = e.category_id';
if ($where) $sql .= ' WHERE ' . implode(' AND ', $where);
$sql .= ' ORDER BY e.created_at DESC LIMIT 200';

$stmt = Db::pdo()->prepare($sql);
$stmt->execute($params);
$rows = $stmt->fetchAll();

$out = array_map(
    fn($r) => Mapper::estate($r, $this->photoUrls((int) $r['id'])),
    $rows
);
return Json::write($response, ['estates' => $out]);
}

public function show(Request $request, Response $response, array $args): Response
{
    $id = (int) $args['id'];
    $stmt = Db::pdo()->prepare(
        'SELECT e.*,
            c.name AS category_name,
            u.email AS owner_email,
            u.phone AS owner_phone,
            u.first_name AS owner_first_name,
            u.last_name AS owner_last_name
        FROM estate e
        JOIN category c ON c.id = e.category_id
        JOIN user u      ON u.id = e.user_id
        WHERE e.id = :id'
    );
    $stmt->execute([':id' => $id]);
    $row = $stmt->fetch();
    if (!$row) {
        return Json::error($response, 'Estate not found', 404);
    }

    return Json::write($response, [
        'estate' => Mapper::estate($row, $this->photoUrls($id)),
    ]);
}

public function create(Request $request, Response $response): Response
{
    $user = $request->getAttribute('user');
    $body = (array) ($request->getParsedBody() ?? []);

    $title      = trim((string) ($body['title'] ?? ''));
    $description = trim((string) ($body['description'] ?? ''));
    $city       = trim((string) ($body['city'] ?? ''));
    $address    = trim((string) ($body['address'] ?? ''));
    $price      = (float) ($body['price'] ?? 0);
    $area       = (float) ($body['area'] ?? 0);
    $categoryId = (int) ($body['categoryId'] ?? 0);
    $lat        = isset($body['lat']) ? (float) $body['lat'] : null;
    $lont       = isset($body['lont']) ? (float) $body['lont'] : null;

    if ($title === '' || $city === '' || $price <= 0 || $area <= 0 || $categoryId <= 0) {
        return Json::error(
            $response,
            'title, city, price, area, categoryId are required',
            422
        );
    }

    $pdo = Db::pdo();
    $pdo->prepare(

```

```

        'INSERT INTO estate
        (category_id, title, description, city, price, area, address, lat, lont, user_id)
        VALUES (:c,:t,:d,:ci,:p,:a,:ad,:la,:lo,:u)'
    )->execute([
        ':c' => $categoryId, ':t' => $title, ':d' => $description,
        ':ci' => $city,       ':p' => $price, ':a' => $area,
        ':ad' => $address,    ':la' => $lat,  ':lo' => $lont,
        ':u' => $user['id'],
    ]);

    // Връщаме новосъздадения имот в същия формат като /api/estates/{id}
    return $this->show($request, $response, ['id' => (string) $pdo->lastInsertId()]);
}

public function delete(Request $request, Response $response, array $args): Response
{
    $user = $request->getAttribute('user');
    $id    = (int) $args['id'];
    $pdo   = Db::pdo();

    $stmt = $pdo->prepare('SELECT user_id FROM estate WHERE id = :id');
    $stmt->execute([':id' => $id]);
    $row = $stmt->fetch();
    if (!$row) {
        return Json::error($response, 'Estate not found', 404);
    }
    if ((int) $row['user_id'] !== (int) $user['id']) {
        return Json::error($response, 'Not allowed', 403);
    }
    $pdo->prepare('DELETE FROM estate WHERE id = :id')->execute([':id' => $id]);
    return Json::write($response, ['ok' => true]);
}

private function photoUrls(int $estateId): array
{
    $base = rtrim((string) ($_ENV['PUBLIC_BASE_URL'] ?? ''), '/');
    $stmt = Db::pdo()->prepare(
        'SELECT file_name FROM estate_photo WHERE estate_id = :id ORDER BY id'
    );
    $stmt->execute([':id' => $estateId]);
    return array_map(
        fn($r) => $base . '/uploads/' . $r['file_name'],
        $stmt->fetchAll()
    );
}
}

```

Ключови концепции

Динамичен WHERE clause

```

$where = [];
$params = [];
if (!empty($q['city'])) {
    $where[] = 'e.city LIKE :city';
    $params[':city'] = '%' . $q['city'] . '%';
}
// ...
if ($where) $sql .= ' WHERE ' . implode(' AND ', $where);

```

Това е стандартен паттерн в PHP за изграждане на SQL заявки с опционални филтри. Всеки филтър добавя:

- Условие в WHERE clause-a.
- Параметър в bind-овете.

prepare + execute с параметрите — никакъв риск от SQL injection.

LIKE с wildcard

```
$params[':city'] = '%' . $q['city'] . '%';
```

% е wildcard. '%Sof%' ще съвпадне с Sofia и Sofia-Bankya. По-приятен опит от exact match.

JOIN-ове в show

```
SELECT e.*, c.name AS category_name, u.email AS owner_email, ...
FROM estate e
JOIN category c ON c.id = e.category_id
JOIN user u     ON u.id = e.user_id
```

В една заявка взимаме данните за имота + името на категорията + цяла информация за собственика. Маргер-ът после ги обвива в правилния JSON.

Връщане на новосъздаден имот

```
return $this->show($request, $response, ['id' => (string) $pdo->lastInsertId()]);
```

След INSERT, директно викаме show() с новото id. Така клиентът получава пълния обект с името на категорията и т.н., готов да го покаже.

Authorization check

```
if ((int) $row['user_id'] !== (int) $user['id']) {
    return Json::error($response, 'Not allowed', 403);
}
```

DELETE минава през AuthMiddleware, така че \$user е сигурно валиден потребител. Допълнително проверяваме че той е **собственикът** на конкретния имот.

photoUrls helper

Снимките са в отделна таблица estate_photo. Тази заявка чете всички файлови имена и ги превръща в пълни URL-и (https://your-domain.tld/uploads/xxx.jpg). PUBLIC_BASE_URL идва от .env.

Стъпка 2 — Регистриране на route-ите

В public/index.php, добави import:

```
use BezBroker\Controller\EstateController;
```

И добави route-овете в \$app->group('/api', ...):

```
// Публични
$g->get('/categories',      [CategoryController::class, 'list']);
$g->get('/estates',         [EstateController::class, 'list']);    // ← нов
$g->get('/estates/{id}',    [EstateController::class, 'show']);    // ← нов

// Защитени
$g->group('', function ($auth) {
    // ... me, logout ...
    $auth->post  ('/estates',      [EstateController::class, 'create']);    // ← нов
    $auth->delete('/estates/{id}', [EstateController::class, 'delete']);    // ← нов
})->add(new AuthMiddleware());
```

Стъпка 3 — Тест

Първо вземи токен от Урок 07:

```
TOKEN=$(curl -s -X POST http://localhost:8000/api/login \
-H "Content-Type: application/json" \
```

```
-d '{"email":"ana@test.bg","password":"123456"}' | grep -oP '"token":'\K[^\"]+')
```

```
echo $TOKEN
```

Създаване на имот:

```
curl -X POST http://localhost:8000/api/estates \
-H "Authorization: Bearer$TOKEN" \
-H "Content-Type: application/json" \
-d '{
  "title": "Двустаен в Лозенец",
  "description": "След ремонт",
  "city": "Sofia",
  "address": "ул. Цар Иван Асен II 24",
  "price": 120000,
  "area": 85,
  "categoryId": 1,
  "lat": 42.6770,
  "lont": 23.3219
}'
```

Очаквай: {estate: {id: 1, ...}}.

Списък:

```
curl http://localhost:8000/api/estates
```

Списък с филтър:

```
curl "http://localhost:8000/api/estates?city=Sof&categoryId=1&maxPrice=200000"
```

Детайл:

```
curl http://localhost:8000/api/estates/1
```

Изтриване (от друг user — трябва да върне 403):

Регистрирай втори user (Boris), вземи неговия token, и:

```
curl -i -X DELETE http://localhost:8000/api/estates/1 \
-H "Authorization: Bearer$BORIS_TOKEN"
# HTTP/1.1 403 – Not allowed
```

С токена на собственика — работи:

```
curl -i -X DELETE http://localhost:8000/api/estates/1 \
-H "Authorization: Bearer$TOKEN"
# {"ok":true}
```

Готови сме

В Урок 10 правим Photo контролера — качване на снимки.

Често срещани грешки

Симптом	Причина
422 при create	Едно от задължителните полета липсва или е невалидно. Виж message-a.
403 при delete	Не си собственикът на имота.
Числата идват като string в Android	Mapper-ът cast-ва с (int) и (float). Провери че ползваш Mapper-a.

Симптом	Причина
null за lat/lont	Не си ги подал в request body-то. Това е валидно — координатите са optional.

Урок 10 — Photo контролер (качване на снимки)

Цел

Покриваме:

- GET /api/estates/{id}/photos — списък с URL-и (публичен)
- POST /api/estates/{id}/photos — multipart upload (защитен, само собственикът)

Как работи multipart upload?

Когато Android клиентът качи снимка, тялото на заявката изглежда така:

```
POST /api/estates/1/photos HTTP/1.1
Content-Type: multipart/form-data; boundary=-----xxx
Authorization: Bearer abc...

-----xxx
Content-Disposition: form-data; name="photos"; filename="photo.jpg"
Content-Type: image/jpeg

<байтове на снимката>
-----xxx--
```

PHP автоматично го парсва — `$_FILES['photos']` (или в Slim:

`$request->getUploadedFiles()['photos']`) ни дава готов обект с името и съдържанието.

Стъпка 1 — PhotoController.php

Създай `api/src/Controller/PhotoController.php`:

```
<?php
declare(strict_types=1);

namespace BezBroker\Controller;

use BezBroker\App\Db;
use BezBroker\App\Json;
use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;
use Psr\Http\Message\UploadedFileInterface;

final class PhotoController
{
    private const ALLOWED = [
        'image/jpeg' => 'jpg',
        'image/png'   => 'png',
        'image/webp'  => 'webp',
    ];

    public function list(Request $request, Response $response, array $args): Response
    {
        $id = (int) $args['id'];
        $stmt = Db::pdo()->prepare(
            'SELECT id, file_name FROM estate_photo WHERE estate_id = :id ORDER BY id'
        );
        $stmt->execute([':id' => $id]);
        $base = rtrim((string) ($_ENV['PUBLIC_BASE_URL'] ?? ''), '/');
        $rows = array_map(fn($r) => [
```

```

        'id' => (int) $r['id'],
        'url' => $base . '/uploads/' . $r['file_name'],
    ], $stmt->fetchAll());

    return Json::write($response, ['photos' => $rows]);
}

public function upload(Request $request, Response $response, array $args): Response
{
    $user      = $request->getAttribute('user');
    $estateId = (int) $args['id'];

    // Провери дали имотът съществува и потребителят е собственикът
    $stmt = Db::pdo()->prepare('SELECT user_id FROM estate WHERE id = :id');
    $stmt->execute([':id' => $estateId]);
    $estate = $stmt->fetch();
    if (!$estate) {
        return Json::error($response, 'Estate not found', 404);
    }
    if ((int) $estate['user_id'] !== (int) $user['id']) {
        return Json::error($response, 'Not allowed', 403);
    }

    // Чети файловете
    $files = $request->getUploadedFiles()['photos'] ?? [];
    if ($files instanceof UploadedFileInterface) {
        $files = [$files]; // ако клиентът качи само 1 файл
    }
    if (!is_array($files) || empty($files)) {
        return Json::error(
            $response,
            'No files provided. Send multipart with field "photos".',
            422
        );
    }

    $uploadsDir = $this->ensureUploadsDir();
    $saved = [];

    foreach ($files as $file) {
        if (!$file instanceof UploadedFileInterface) continue;
        if ($file->getError() !== UPLOAD_ERR_OK) continue;

        $mime = (string) $file->getClientMediaType();
        if (!isset(self::ALLOWED[$mime])) {
            return Json::error($response, "Unsupported file type: $mime", 415);
        }
        $ext = self::ALLOWED[$mime];
        $name = sprintf(
            '%d_%s.%s',
            $estateId,
            bin2hex(random_bytes(8)),
            $ext
        );
        $file->moveTo($uploadsDir . DIRECTORY_SEPARATOR . $name);

        Db::pdo()->prepare(
            'INSERT INTO estate_photo (estate_id, file_name) VALUES (:e, :n)'
        )->execute([':e' => $estateId, ':n' => $name]);

        $saved[] = $name;
    }

    $base = rtrim((string) ($_ENV['PUBLIC_BASE_URL'] ?? ''), '/');
    return Json::write($response, [
        'uploaded' => count($saved),
    ]);
}

```

```

        'urls'      => array_map(fn($n) => $base . '/uploads/' . $n, $saved),
    ], 201);
}

private function ensureUploadsDir(): string
{
    $sub = trim((string) ($_ENV['UPLOADS_DIR'] ?? 'uploads'), '/');
    $dir = realpath(__DIR__ . '/../../public');
    if ($dir === false) {
        throw new \RuntimeException('public/ directory not found');
    }
    $full = $dir . DIRECTORY_SEPARATOR . $sub;
    if (!is_dir($full) && !mkdir($full, 0775, true) && !is_dir($full)) {
        throw new \RuntimeException("Cannot create uploads directory: $full");
    }
    return $full;
}
}

```

Концепции

Whitelist на MIME типовете

```

private const ALLOWED = [
    'image/jpeg' => 'jpg',
    'image/png'  => 'png',
    'image/webp' => 'webp',
];

```

Защо whitelist а не blacklist? Защото blacklist е лесен за заобикаляне. Whitelist казва "точно тези типове са разрешени, всичко друго не".

'image/heic' (iPhone snapshots) ще получи 415 Unsupported Media Type. Това е приемливо ограничение.

Случайно име на файла

```

$name = sprintf(
    '%d_%s.%s',
    $estateId,
    bin2hex(random_bytes(8)),
    $ext
);
// Например: "5_a3f2b7c4d8e9f1a0.jpg"

```

Защо случайно? Защото:

- Ако пазим оригиналното име, два потребителя качват IMG_0001.jpg и един от тях ще се запише върху другия.
- Random името е guessable — никой не може да изтегли чужди снимки като отгатне URL-а.

moveTo() **ВМЕСТО** move_uploaded_file()

PSR-7 интерфейсът UploadedFileInterface има moveTo(\$path) — върши същата работа но е по-преносим (работи и за тестване с Slim\Psr7\UploadedFile).

Authorization check

Двойна проверка:

1. AuthMiddleware — потребителят е логнат (има валиден токен).
2. Тук — той е **собственикът** на имота.

Без втората проверка, Иван би могъл да качи снимки в обявата на Ана.

Стъпка 2 — Регистриране на route-ите

В `public/index.php`:

```
use BezBroker\Controller\PhotoController;

// ...

// Public
$g->get('/estates/{id}/photos', [PhotoController::class, 'list']);

// Authenticated
$auth->post('/estates/{id}/photos', [PhotoController::class, 'upload']);
```

Стъпка 3 — Папка uploads/

Не е критично — `ensureUploadsDir()` я създава автоматично при първи upload. Но за production се препоръчва ръчно:

```
mkdir -p api/public/uploads
chmod 775 api/public/uploads
```

(`chmod` работи на Linux/Plesk. На Windows не е нужно — папките са с пълни права.)

Стъпка 4 — Тест

Имай токен от Урок 07 и имот от Урок 09 (`estate_id = 1`).

Качване на 1 снимка:

```
curl -X POST http://localhost:8000/api/estates/1/photos \
-H "Authorization: Bearer$TOKEN" \
-F "photos=@C:/Users/PC/Pictures/test.jpg"
```

Очаквай:

```
{
  "uploaded": 1,
  "urls": ["http://10.0.2.2:8000/uploads/1_a3f2b7c4d8e9f1a0.jpg"]
}
```

-F е curl флагът за multipart form data. @path казва "качи съдържанието на този файл".

Качване на няколко снимки:

```
curl -X POST http://localhost:8000/api/estates/1/photos \
-H "Authorization: Bearer$TOKEN" \
-F "photos=@C:/photos/a.jpg" \
-F "photos=@C:/photos/b.jpg" \
-F "photos=@C:/photos/c.jpg"
```

Списък:

```
curl http://localhost:8000/api/estates/1/photos
```

Проверка на снимките през браузъра

Отвори някоя от URL-ите в браузъра (например `http://localhost:8000/uploads/1_xxx.jpg`) — трябва да видиш снимката.

Важно: PHP вграденият сървър няма `.htaccess rewrite`-ове и сам сервира файлове от document root-а (`public/`). На Plesk-а `.htaccess` ще се грижи за това (виж Урок 12).

Стъпка 5 — Тест на грешки

Грешен файл тип:

```
curl -i -X POST http://localhost:8000/api/estates/1/photos \
  -H "Authorization: Bearer$TOKEN" \
  -F "photos=@document.pdf"
# 415 – Unsupported file type
```

Не съм собственикът:

```
curl -i -X POST http://localhost:8000/api/estates/1/photos \
  -H "Authorization: Bearer$BORIS_TOKEN" \
  -F "photos=@image.jpg"
# 403 – Not allowed
```

Несъществуващ имот:

```
curl -i -X POST http://localhost:8000/api/estates/9999/photos \
  -H "Authorization: Bearer$TOKEN" \
  -F "photos=@image.jpg"
# 404 – Estate not found
```

Готови сме

В Урок 11 правим Offer и Message контролерите.

Често срещани грешки

Симптом	Причина
422 "No files provided"	Полето в multipart-a е с грешно име. Трябва да е photos (виж name="photos" в Content-Disposition).
415 при iPhone снимки	iPhone праща image/heic или image/heif. Можеш да ги добавиш в ALLOWED, но първо ще трябва да конвертираш.
413 Payload Too Large	PHP конфигурация. В php.ini: upload_max_filesize = 10M, post_max_size = 12M. Рестартирай PHP сървъра.
Снимката се пази, но GET връща 404	PUBLIC_BASE_URL в .env сочи към грешен адрес. Провери.
Permission denied при mkdir	На Linux/Plesk: chmod 775 public/ или промени собственика на папката.

Урок 11 — Offer и Message контролери

Цел

Покриваме последните два endpoint-a:

Offers:

- POST /api/offers — нова оферта (защитен)
- GET /api/offers — моите оферти (като купувач или като продавач)
- POST /api/offers/{id}/state — accept / reject (само продавачът)

Messages:

- GET /api/offers/{id}/messages?since=N — съобщения в чат
- POST /api/offers/{id}/messages — нов message

Стъпка 1 — OfferController.php

Създай api/src/Controller/OfferController.php:

```
<?php
declare(strict_types=1);

namespace BezBroker\Controller;

use BezBroker\App\Db;
use BezBroker\App\Json;
use BezBroker\App\Mapper;
use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;

final class OfferController
{
    public function create(Request $request, Response $response): Response
    {
        $user = $request->getAttribute('user');
        $body = (array) ($request->getParsedBody() ?? []);

        $estateId = (int) ($body['estateId'] ?? 0);
        $sum = (float) ($body['sum'] ?? 0);
        if ($estateId <= 0 || $sum <= 0) {
            return Json::error($response, 'estateId and sum are required.', 422);
        }

        $pdo = Db::pdo();
        $stmt = $pdo->prepare('SELECT user_id FROM estate WHERE id = :id');
        $stmt->execute([':id' => $estateId]);
        $estate = $stmt->fetch();
        if (!$estate) {
            return Json::error($response, 'Estate not found', 404);
        }
        if ((int) $estate['user_id'] === (int) $user['id']) {
            return Json::error(
                $response,
                'You cannot make an offer on your own estate',
                422
            );
        }

        $pdo->prepare(
            'INSERT INTO offer (sum, offered_by_user, offered_for_estate, state)
```

```

        VALUES (:s, :u, :e, "pending")'
    )->execute([':s' => $sum, ':u' => $user['id'], ':e' => $estateId]);

    return Json::write($response, [
        'offer' => Mapper::offer($this->loadOffer((int) $pdo->lastInsertId())),
    ], 201);
}

public function mine(Request $request, Response $response): Response
{
    $user = $request->getAttribute('user');
    $stmt = Db::pdo()->prepare(
        'SELECT o.*,
            e.title          AS estate_title,
            e.user_id        AS estate_owner_id,
            buyer.first_name AS buyer_first_name,
            buyer.last_name  AS buyer_last_name,
            seller.first_name AS seller_first_name,
            seller.last_name  AS seller_last_name
        FROM offer o
        JOIN estate e  ON e.id = o.offered_for_estate
        JOIN user buyer ON buyer.id = o.offered_by_user
        JOIN user seller ON seller.id = e.user_id
        WHERE o.offered_by_user = :u OR e.user_id = :u
        ORDER BY o.created_at DESC'
    );
    $stmt->execute([':u' => $user['id']]);
    $rows = array_map(fn($r) => Mapper::offer($r), $stmt->fetchAll());
    return Json::write($response, ['offers' => $rows]);
}

public function changeState(Request $request, Response $response, array $args): Response
{
    $user = $request->getAttribute('user');
    $id  = (int) $args['id'];
    $body = (array) ($request->getParsedBody() ?? []);
    $newState = $body['state'] ?? '';
    if (!in_array($newState, ['accepted', 'rejected'], true)) {
        return Json::error($response, 'state must be "accepted" or "rejected"', 422);
    }

    $pdo = Db::pdo();
    $stmt = $pdo->prepare(
        'SELECT o.id, e.user_id AS seller_id
        FROM offer o
        JOIN estate e ON e.id = o.offered_for_estate
        WHERE o.id = :id'
    );
    $stmt->execute([':id' => $id]);
    $row = $stmt->fetch();
    if (!$row) {
        return Json::error($response, 'Offer not found', 404);
    }
    if ((int) $row['seller_id'] !== (int) $user['id']) {
        return Json::error(
            $response,
            'Only the estate owner can change the offer state',
            403
        );
    }

    $pdo->prepare('UPDATE offer SET state = :s WHERE id = :id')
    ->execute([':s' => $newState, ':id' => $id]);

    return Json::write($response, [
        'offer' => Mapper::offer($this->loadOffer($id)),
    ], 201);
}

```

```

    });
}

private function loadOffer(int $id): array
{
    $stmt = Db::pdo()->prepare('SELECT * FROM offer WHERE id = :id');
    $stmt->execute([':id' => $id]);
    return $stmt->fetch() ?: [];
}
}

```

Бизнес правила:

- В create не може да направиш оферта на свой имот (422).
- В changeState само собственикът на имота може да приема/отхвърля.
- В mine връщаме оферти, в които участваме като купувач **или** като продавач — затова WHERE o.offered_by_user = :u OR e.user_id = :u.

Стъпка 2 — MessageController.php

Създай api/src/Controller/MessageController.php:

```

<?php
declare(strict_types=1);

namespace BezBroker\Controller;

use BezBroker\App\Db;
use BezBroker\App\Json;
use BezBroker\App\Mapper;
use Psr\Http\Message\ResponseInterface as Response;
use Psr\Http\Message\ServerRequestInterface as Request;

final class MessageController
{
    public function list(Request $request, Response $response, array $args): Response
    {
        $user = $request->getAttribute('user');
        $offerId = (int) $args['id'];
        $since = (int) ($request->getQueryParams()['since'] ?? 0);

        if (!$this->canSee($offerId, (int) $user['id'])) {
            return Json::error($response, 'Not allowed', 403);
        }

        $stmt = Db::pdo()->prepare(
            'SELECT * FROM message
            WHERE offer_id = :o AND id > :since
            ORDER BY id ASC'
        );
        $stmt->execute([':o' => $offerId, ':since' => $since]);
        $rows = array_map(fn($r) => Mapper::message($r), $stmt->fetchAll());
        return Json::write($response, ['messages' => $rows]);
    }

    public function create(Request $request, Response $response, array $args): Response
    {
        $user = $request->getAttribute('user');
        $offerId = (int) $args['id'];
        $body = (array) ($request->getParsedBody() ?? []);
        $text = trim((string) ($body['text'] ?? ''));
        if ($text === '') {
            return Json::error($response, 'text is required', 422);
        }
    }
}

```

```

    $other = $this->otherPartyOrNull($offerId, (int) $user['id']);
    if ($other === null) {
        return Json::error($response, 'Not allowed', 403);
    }

    $pdo = Db::pdo();
    $pdo->prepare(
        'INSERT INTO message (text, from_user, to_user, offer_id)
        VALUES (:t, :f, :u, :o)'
    )->execute([
        ':t' => $text,
        ':f' => $user['id'],
        ':u' => $other,
        ':o' => $offerId,
    ]);

    $id = (int) $pdo->lastInsertId();
    $stmt = $pdo->prepare('SELECT * FROM message WHERE id = :id');
    $stmt->execute([':id' => $id]);
    return Json::write($response, [
        'message' => Mapper::message($stmt->fetch() ?: []),
    ], 201);
}

/**
 * Връща id-то на ДРУГАТА страна в офертата (купувач или продавач),
 * ако текущият user участва. Иначе null.
 */
private function otherPartyOrNull(int $offerId, int $userId): ?int
{
    $stmt = Db::pdo()->prepare(
        'SELECT o.offered_by_user AS buyer_id, e.user_id AS seller_id
        FROM offer o JOIN estate e ON e.id = o.offered_for_estate
        WHERE o.id = :id'
    );
    $stmt->execute([':id' => $offerId]);
    $row = $stmt->fetch();
    if (!$row) return null;
    $buyer = (int) $row['buyer_id'];
    $seller = (int) $row['seller_id'];
    if ($userId === $buyer) return $seller;
    if ($userId === $seller) return $buyer;
    return null;
}

private function canSee(int $offerId, int $userId): bool
{
    return $this->otherPartyOrNull($offerId, $userId) !== null;
}
}

```

Какво да забележим:

Polling c since

```

$since = (int) ($request->getQueryParams()['since'] ?? 0);
// ...
'WHERE offer_id = :o AND id > :since'

```

Клиентът пази `lastId` и пита "дай ми всички съобщения с `id > X`". Така polling-ът на всеки 3 секунди не дърпа всеки път целия чат.

При първото викане `since=0` → връща всички. На следващите — само новите.

Авторизация през otherPartyOrNull

Един метод върши две неща:

1. Връща null ако userId не участва в офертата (за блокиране).
2. Иначе връща id-то на отсрещната страна (за to_user колоната).

Тогава canSee е просто != null.

from_user и to_user в message

```
from_user INT NOT NULL,  
to_user   INT NOT NULL,
```

Защо пазим **двете**, а не само едното? Защото отсрещната страна се определя от офертата (sender или owner на имота), и така е по-удобно за бъдещи feature-и (например "брой непрочетени съобщения, изпратени към мен").

Стъпка 3 — Регистриране на route-ите

В public/index.php:

```
use BezBroker\Controller\OfferController;  
use BezBroker\Controller\MessageController;  
  
// Authenticated:  
$auth->post('/offers', [OfferController::class, 'create']);  
$auth->get ('/offers', [OfferController::class, 'mine']);  
$auth->post('/offers/{id}/state', [OfferController::class, 'changeState']);  
$auth->get ('/offers/{id}/messages', [MessageController::class, 'list']);  
$auth->post('/offers/{id}/messages', [MessageController::class, 'create']);
```

Всички са защитени (под AuthMiddleware-a).

Стъпка 4 — Краен вид на index.php

В този момент твоят public/index.php трябва да изглежда нещо като:

```
<?php  
declare(strict_types=1);  
  
use Dotenv\Dotenv;  
use Slim\Factory\AppFactory;  
use BezBroker\App\Json;  
use BezBroker\Middleware\AuthMiddleware;  
use BezBroker\Middleware\CorsMiddleware;  
use BezBroker\Middleware\JsonBodyMiddleware;  
use BezBroker\Controller\AuthController;  
use BezBroker\Controller\CategoryController;  
use BezBroker\Controller\EstateController;  
use BezBroker\Controller\PhotoController;  
use BezBroker\Controller\OfferController;  
use BezBroker\Controller\MessageController;  
  
require __DIR__ . '/../vendor/autoload.php';  
Dotenv::createImmutable(__DIR__ . '..')->safeLoad();  
  
$app = AppFactory::create();  
  
$app->addBodyParsingMiddleware();  
$app->add(new JsonBodyMiddleware());  
$app->add(new CorsMiddleware());  
  
$errorMiddleware = $app->addErrorMiddleware(false, true, true);  
$errorMiddleware->setDefaultErrorHandler(function ($request, \Throwable $exception) {
```

```

$status = 500;
if ($exception instanceof \Slim\Exception\HttpNotFoundException) $status = 404;
if (method_exists($exception, 'getCode')
    && $exception->getCode() >= 400 && $exception->getCode() < 600) {
    $status = $exception->getCode();
}
$payload = ['error' => true, 'message' => $exception->getMessage() ?: 'Server error'];
$res = (new \Slim\Psr7\Response())->withStatus($status);
$res->getBody()->write(json_encode($payload, JSON_UNESCAPED_UNICODE));
return $res->withHeader('Content-Type', 'application/json');
});

$app->get('/', fn($req, $res) => Json::write($res, ['name' => 'BezBroker API', 'ok' => true]));

$app->group('/api', function ($g) {
    $g->post('/register', [AuthController::class, 'register']);
    $g->post('/login', [AuthController::class, 'login']);

    $g->get('/categories', [CategoryController::class, 'list']);
    $g->get('/estates', [EstateController::class, 'list']);
    $g->get('/estates/{id}', [EstateController::class, 'show']);
    $g->get('/estates/{id}/photos', [PhotoController::class, 'list']);

    $g->group('', function ($auth) {
        $auth->get('/me', [AuthController::class, 'me']);
        $auth->post('/logout', [AuthController::class, 'logout']);

        $auth->post('/estates', [EstateController::class, 'create']);
        $auth->delete('/estates/{id}', [EstateController::class, 'delete']);
        $auth->post('/estates/{id}/photos', [PhotoController::class, 'upload']);

        $auth->post('/offers', [OfferController::class, 'create']);
        $auth->get('/offers', [OfferController::class, 'mine']);
        $auth->post('/offers/{id}/state', [OfferController::class, 'changeState']);

        $auth->get('/offers/{id}/messages', [MessageController::class, 'list']);
        $auth->post('/offers/{id}/messages', [MessageController::class, 'create']);
    })->add(new AuthMiddleware());
});

$app->options('/{routes:.+}', fn($req, $res) => $res);
$app->run();

```

Стъпка 5 — Тест на пълния flow

Регистрирай 2 потребителя (Ана — seller, Boris — buyer). Вземи токените им.

```

# Boris прави оферта
curl -X POST http://localhost:8000/api/offers \
  -H "Authorization: Bearer$BORIS_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"estateId":1,"sum":100000}'
# {"offer":{"...}}

# Boris вижда офертата си
curl http://localhost:8000/api/offers \
  -H "Authorization: Bearer$BORIS_TOKEN"

# Ана също я вижда (като продавач)
curl http://localhost:8000/api/offers \
  -H "Authorization: Bearer$ANA_TOKEN"

# Boris пише съобщение
curl -X POST http://localhost:8000/api/offers/1/messages \
  -H "Authorization: Bearer$BORIS_TOKEN" \

```

```

-H "Content-Type: application/json" \
-d '{"text":"Здравейте!"}'

# Ана чете съобщенията
curl "http://localhost:8000/api/offers/1/messages?since=0" \
-H "Authorization: Bearer$ANA_TOKEN"
# {"messages":[{"...}]}

# Ана отговаря
curl -X POST http://localhost:8000/api/offers/1/messages \
-H "Authorization: Bearer$ANA_TOKEN" \
-H "Content-Type: application/json" \
-d '{"text":"Здравейте! Кога може да го видите?"}'

# Boris приема... всъщност не – само продавачът може!
curl -i -X POST http://localhost:8000/api/offers/1/state \
-H "Authorization: Bearer$BORIS_TOKEN" \
-H "Content-Type: application/json" \
-d '{"state":"accepted"}'
# 403 – Only the estate owner can change the offer state

# Ана приема
curl -X POST http://localhost:8000/api/offers/1/state \
-H "Authorization: Bearer$ANA_TOKEN" \
-H "Content-Type: application/json" \
-d '{"state":"accepted"}'
# {"offer":{"...","state":"accepted"}}

```

Готови сме

API-то е **функционално готово**. В Урок 12 правим финален тест + качване на Plesk сървър.

Често срещани грешки

Симптом	Причина
422 "You cannot make an offer on your own estate"	Логнат си като собственика на имота. Логни се с друг user.
403 при changeState	Не си собственикът.
403 при messages	Не участваш в офертата.
Празен списък със съобщения, въпреки че имаш	Може since параметърът да е твърде голям. Опитай ?since=0.

Урок 12 — Качване на Plesk + финален тест

Цел

Да хостнем API-то на реален сървър, така че Android приложението да може да го достигне отвсякъде (а не само от компютъра на който е развит).

Този урок е практически чеклист. Очакваното време: ~30 минути.

Какво ще ти трябва

- Достъп до Plesk панел.
- Домейн или поддомейн (например `api.your-domain.tld`).
- PHP 8.1+ активиран в Plesk PHP Settings.

Стъпка 1 — База данни на Plesk

В Plesk → **Databases** → **Add Database**:

- Name: `bezbroker`
- User: `bezbroker`
- Password: силна, запиши я

В phpMyAdmin → избери `bezbroker` → **Import** → `api/sql/schema.sql` → **Go**.

Резултат: 7 таблици + 5 категории.

Стъпка 2 — Качване на файловете

Вариант А — Git (препоръчително)

Plesk → **Git** → **Add Repository** → **External**:

- URL: твоя Git remote.
- Deployment path: `httpdocs`.

При всеки `git push` Plesk автоматично pull-ва.

Вариант В — ръчно FTP/File Manager

Качи цялата папка `api/` като съдържание на `httpdocs/`. Тоест:

```
httpdocs/  
├── public/  
├── src/  
├── sql/  
├── composer.json  
└── .env.example
```

Не качвай:

- `vendor/` (ще се изтегли от Composer на сървъра).
- `.env` (ще го направим там, с production credentials).
- `public/uploads/` (ще се пълни от потребителите).

Стъпка 3 — Composer install на сървъра

Plesk → **Domains** → `your-domain` → **Composer** → намира `composer.json` → **Install**.

Алтернативно от SSH:

```
cd ~/httpdocs
composer install --no-dev --optimize-autoloader
```

Резултат: появява се папка vendor/.

Стъпка 4 — Production .env

В File Manager-а копирай .env.example → .env и редактирай:

```
DB_HOST=localhost
DB_NAME=bezbroker
DB_USER=bezbroker
DB_PASS=твоята-парола-от-стъпка-1

UPLOADS_DIR=uploads
PUBLIC_BASE_URL=https://your-domain.tld
```

Стъпка 5 — Document root → httpdocs/public

Критично. Без този ред source кода на API-то ще се виждат публично.

Plesk → **Hosting & DNS** → **Hosting Settings**:

- **Document Root:** httpdocs/public

Запази.

Стъпка 6 — Папка за uploads

```
cd ~/httpdocs/public
mkdir -p uploads
chmod 775 uploads
```

Или от File Manager: десен бутон → Change Permissions → 775.

Стъпка 7 — HTTPS

Plesk → **SSL/TLS Certificates** → **Install free Let's Encrypt** → ☐ домейна → **Install**.

После: **Hosting Settings** → ☐ "Permanent SEO-safe 301 redirect from HTTP to HTTPS".

Стъпка 8 — Тест със curl

```
# 1. Health check
curl https://your-domain.tld/
# {"name":"BezBroker API","ok":true}

# 2. Категории
curl https://your-domain.tld/api/categories
# {"categories":[{"id":1,"name":"Apartment"},...]}

# 3. Регистрация
curl -X POST https://your-domain.tld/api/register \
  -H "Content-Type: application/json" \
  -d '{"email":"test@a.bg","password":"123456","firstName":"Test","lastName":"User","isSeller":true}'
```

Ако всичките минават → API-то е готово!

Стъпка 9 — Свържи Android приложението

В Android Studio отвори `ApiClient.java`:

```
public static final String BASE_URL = "https://bezbroker-api.onrender.com";
```

Замени с:

```
public static final String BASE_URL = "https://your-domain.tld";
```

Rebuild → Run. Регистрирай се от приложението — трябва да отива към твоя сървър.

Финален чеклист

- [] `https://your-domain.tld/` връща `{"ok":true}`
- [] `https://your-domain.tld/api/categories` връща 5 категории
- [] Регистрация работи (връща token + user)
- [] Login работи
- [] `/api/me` с токен връща user-a
- [] Photo upload работи (опитай `curl -F`)
- [] HTTPS активен (зелено катинарче в браузъра)
- [] Папка `public/uploads/` съществува, права 775
- [] `BASE_URL` в `ApiClient.java` обновен

Troubleshooting

Симптом	Какво да проверяваш
500 Internal Server Error навсякъде	Plesk PHP версия да е 8.1+. Включени разширения: <code>pdo_mysql</code> , <code>mbstring</code> , <code>fileinfo</code> , <code>openssl</code> .
404 на всички пътища	Document root не сочи към <code>public/</code> . Или <code>.htaccess</code> не се прилага (Apache modules → <code>mod_rewrite</code> да е включен).
500 само при <code>/api/...</code>	<code>vendor/</code> го няма — <code>composer install</code> не е пуснат.
Connection refused от приложението	HTTPS сертификат не работи. Виж дали <code>https://...</code> отваря в браузъра.
Photo upload 413 Payload Too Large	Plesk → PHP Settings : <code>upload_max_filesize = 10M</code> , <code>post_max_size = 12M</code> .
Authorization header missing	nginx не препраща Authorization header-a. На nginx vhost-a: <code>fastcgi_pass_header Authorization</code> ; На Apache + <code>.htaccess</code> работи без допълнителна настройка.

Поздравления!

API-то е production-ready. С това приключваме Част 1.

Можеш да продължиш с **Част 2 — Разработка на мобилното приложение**.

Идеи за надграждане (за по-напреднали часове)

- **Admin потребител** — добави `role='admin'` и нови endpoint-и за модериране.
- **Pagination** — `?offset=N&limit=20` за `/api/estates`.
- **Геопретърсване** — `?lat=42.7&lon=23.3&radius=5` (радиус в км).
- **Кеширане** — Redis за често исканите endpoint-и.
- **Rate limiting** — Slim middleware против brute-force атаки.
- **JWT** вместо токени в база — без DB lookup при всеки request.
- **Real-time чат** с WebSocket вместо polling — Ratchet библиотека.